

Competitive Programming

Section 20: 0-1 BFS And Shortest Path Variants

Competitive Programming Course

2026-07-05

Table of contents

1	Why 0-1 BFS exists	3
2	Recognition	4
3	Deque primer	5
4	0-1 BFS template	6
5	The invariant	7
6	Worked example	8
7	Grid modeling	9
8	Complexity	10
9	Common mistakes	11
10	Local exercises	12
10.1	A. Binary Weight Routes	12
10.2	B. Conveyor Grid	12
10.3	C. Warp Maze	12
11	Submission checklist	13

1 Why 0-1 BFS exists

Section 19 used Dijkstra for nonnegative weighted shortest paths. If all edge weights are only 0 or 1, we can do better than a heap.

The idea:

```
cost 0 edge: new state has the same distance, so process it very soon  
cost 1 edge: new state is one layer later, so process it after current layer
```

A deque gives exactly these two choices:

```
appendleft for cost 0  
append for cost 1
```

2 Recognition

Use 0-1 BFS when:

Statement signal	Tool
every edge cost is 0 or 1	0-1 BFS
free moves plus paid moves	0-1 BFS
minimize number of direction changes	0-1 BFS
walking costs 0 and warping costs 1	0-1 BFS

Do not use it when weights are arbitrary nonnegative numbers. Use Dijkstra.

3 Deque primer

A deque supports both ends:

```
from collections import deque
dq.appendleft(x)
dq.append(x)
x = dq.popleft()
```

C++:

```
deque<int> dq;
dq.push_front(x);
dq.push_back(x);
x = dq.front();
dq.pop_front();
```

4 0-1 BFS template

```
dist[start] = 0
deque = [start]

while deque not empty:
    u = pop front
    for edge u -> v with weight w in {0,1}:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            if w == 0:
                push v to front
            else:
                push v to back
```

This is Dijkstra specialized to two possible edge weights.

5 The invariant

The deque is ordered by distance almost like a priority queue:

```
states near the front have the smallest known distance  
cost-0 relaxations stay at the front distance  
cost-1 relaxations go to the next distance layer
```

So when a state is popped after its best distance has stabilized, all cheaper ways to reach it have already had a chance to act.

6 Worked example

Edges:

```
1 -> 2 cost 1
1 -> 3 cost 0
3 -> 2 cost 0
2 -> 4 cost 1
3 -> 5 cost 1
5 -> 4 cost 0
```

From 1:

```
pop 1
  push 2 to back with dist 1
  push 3 to front with dist 0
pop 3
  improve 2 to dist 0, push front
  set 5 to dist 1, push back
pop 2
  set 4 to dist 1
```

The answer to 4 is 1.

7 Grid modeling

For Conveyor Grid:

```
state = cell  
edge = move to a neighboring cell  
cost = 0 if movement follows cell arrow, else 1
```

For Warp Maze:

```
walk to four-neighbor open cell: cost 0  
warp within 5x5 square: cost 1
```

Both are standard 0-1 BFS once the states and edge costs are defined.

8 Complexity

Every successful relaxation pushes a state into the deque. With adjacency lists:

```
Time:   $O(V + E)$   
Memory:  $O(V + E)$ 
```

For a grid with constant outgoing moves, this is linear in the number of cells. For Warp Maze, each cell checks at most 4 walking moves and 25 warp moves, still constant per cell.

9 Common mistakes

Using BFS and treating cost-0 and cost-1 edges equally.
Using Dijkstra correctly but more slowly when 0-1 BFS is intended.
Pushing cost-0 edges to the back.
Forgetting to relax a state if a better distance appears later.
Using 0-1 BFS on weights larger than 1.

10 Local exercises

10.1 A. Binary Weight Routes

Pure graph 0-1 BFS. Run once from the source and answer all queries.

10.2 B. Conveyor Grid

Each cell's arrow creates one cost-0 outgoing direction and up to three cost-1 directions.

10.3 C. Warp Maze

Walking is free, warping costs one. The result is the minimum number of warps.

11 Submission checklist

```
Are all edge weights exactly 0 or 1?  
Did I use a deque?  
Did I push 0-cost relaxations to the front?  
Did I push 1-cost relaxations to the back?  
For grids, did I check bounds and walls before relaxing?
```